Лабораторная работа № 4.

Линейная алгебра

Дагделен Зейнап Реджеповна

В данном отчёте представлены результаты выполнения лабораторной работы по теме «Линейная алгебра» с использованием языка программирования Julia.

Описаны использованные методы работы с матрицами и векторами, факторизации, спектрального разложения и модели Леонтьева. Полученные экспериментальные данные и их анализ подтверждают эффективность специализированных пакетов Julia.

Сделаны выводы о достижении поставленной цели.

Содержание

1 Введение **6**
1.1 Актуальность 6
1.2 Цель работы 6
1.3 Задачи 6
1.4 Объект и предмет исследования 7
1.5 Методы исследования 7

2 Теоретическая часть **8**

3 Материалы и методы **10**

4 Выполнение лабораторной работы **11**
4.1 Примеры из раздела 4.2 11
4.2 Задания для самостоятельной работы 15

5 Результаты **24**

6 Обсуждение результатов **26**

7 Заключение **27**

Список литературы **28**

Список иллюстраций

4.1 Поэлементные операции и операции над матрицами 12
4.2 Нормы, расстояния и умножение матриц 13
4.3 Факторизации и спектральное разложение 14
4.4 Оценка эффективности (BenchmarkTools) 15
4.5 Произведение векторов (4.4.1) 16
4.6 Системы линейных уравнений с двумя неизвестными (4.4.2) 17
4.7 Системы линейных уравнений с тремя неизвестными (4.4.2) 18
4.8 Приведение матриц 2x2 к диагональному виду (4.4.3) 19
4.9 Приведение матрицы 3x3 к диагональному виду (4.4.3) 20
4.10 Вычисления (степени и корни) (4.4.3) 21
4.11 Собственные значения и специальные матрицы (4.4.3) 22
4.12 Оценка эффективности умножения матриц (4.4.3) 22
4.13 Проверка продуктивности матриц по определению (4.4.4) 23
4.14 Проверка продуктивности матриц по критериям (4.4.4) 23

Список таблиц

1 Введение

1.1 Актуальность

Линейная алгебра является фундаментом для множества областей современной науки и техники: от машинного обучения и компьютерной графики до экономического моделирования и квантовой механики. Эффективная реализация операций над матрицами и векторами критически важна для производительности вычислительных систем. Язык Julia, обладающий высокой скоростью и удобным синтаксисом, предоставляет мощные инструменты для решения задач линейной алгебры.

1.2 Цель работы

Основной целью работы является изучение возможностей специализированных пакетов Julia для выполнения и оценки эффективности операций над объектами линейной алгебры.

1.3 Задачи

Для достижения поставленной цели необходимо решить следующие задачи:

1. Освоить базовые поэлементные операции над многомерными массивами.
2. Изучить операции транспонирования, нахождения следа, ранга, определителя и обратной матрицы.

3. Научиться вычислять нормы и расстояния между векторами, а также углы между ними.
4. Освоить матричное умножение и скалярное произведение.
5. Изучить факторизации (LU, QR) и спектральное разложение матриц.
6. Провести оценку эффективности вычислений с использованием пакета `BenchmarkTools`.
7. Выполнить задания для самостоятельной работы, включая решение СЛАУ, диагонализацию, вычисление корней и степеней матриц.
8. Реализовать проверку продуктивности матриц в модели Леонтьева по различным критериям.

1.4 Объект и предмет исследования

Объект исследования: процессы выполнения операций линейной алгебры в языке программирования Julia. **Предмет исследования:** эффективность и особенности реализации матричных и векторных вычислений с использованием стандартных библиотек и специализированных структур данных.

1.5 Методы исследования

В работе использовались следующие методы:

- **Экспериментальный:** выполнение кода на языке Julia и анализ получаемых результатов.
- **Сравнительный:** оценка скорости выполнения операций с помощью пакета `BenchmarkTools` для различных типов матриц.
- **Аналитический:** проверка математических свойств (например, ортогональности матриц, продуктивности матриц по критериям).

2 Теоретическая часть

В данной работе используются следующие основные понятия и операции линейной алгебры [2]:

1. **Поэлементные операции** над массивами: суммирование, умножение, нахождение среднего значения. Они применяются к каждому элементу независимо.
2. **Транспонирование** — операция, при которой строки матрицы становятся её столбцами (и наоборот).
3. **След матрицы** — сумма элементов главной диагонали.
4. **Ранг матрицы** — максимальное число линейно независимых строк (или столбцов).
5. **Определитель** — скалярная величина, характеризующая линейное преобразование.
6. **Обратная матрица** A^{-1} — матрица, произведение которой на исходную A даёт единичную матрицу I.
7. **Норма вектора** — длина вектора в евклидовом или другом пространстве.
8. **Скалярное произведение** — операция над двумя векторами, результатом которой является число.
9. **Факторизация** — разложение матрицы на произведение матриц специального вида (LU, QR).
10. **Спектральное разложение** — представление матрицы в виде $A = V\Lambda V^{-1}$, где Λ — диагональная матрица собственных значений, V — матрица

собственных векторов.

11. **Модель Леонтьева** — модель межотраслевого баланса, где матрица A называется продуктивной, если уравнение $x - Ax = y$ имеет неотрицательное решение x при любом неотрицательном y. Критерии продуктивности: матрица $(E - A)^{-1}$ неотрицательна, либо все собственные значения A по модулю меньше 1.

Все программные примеры выполнялись на языке Julia с использованием стандартных библиотек. Синтаксис языка, работа с массивами и специализированными матричными структурами (например, `Symmetric`, `Diagonal`) подробно описаны в официальной документации [1].

3 Материалы и методы

Программное обеспечение: язык программирования Julia (версия 1.x) и следующие пакеты:

- `LinearAlgebra` — для выполнения основных операций линейной алгебры.
- `Statistics` — для вычисления средних значений.
- `Random` — для генерации случайных чисел и матриц.
- `BenchmarkTools` — для оценки времени выполнения операций (`@btime`).

Оборудование: персональный компьютер с установленной средой разработки (например, Jupyter Notebook или REPL Julia).

Последовательность действий:

1. Подключение необходимых пакетов.
2. Выполнение примеров из раздела 4.2 методических указаний с фиксацией результатов в скриншотах.
3. Выполнение заданий для самостоятельной работы (пункты 4.4.1 – 4.4.4).
4. Оценка эффективности операций с помощью бенчмаркинга.
5. Оформление отчёта с описанием каждого шага и приложением скриншотов.

4 Выполнение лабораторной работы

В ходе работы были выполнены следующие действия, соответствующие разделам методических указаний.

4.1 Примеры из раздела 4.2

4.1.1 Поэлементные операции и операции над матрицами

На рис. 4.1 представлен код и результаты выполнения базовых поэлементных операций над матрицей 4x3 (сумма, произведение, среднее), а также операции транспонирования, нахождения следа, ранга, определителя и обратной матрицы для матрицы 4x4.

[3]:
```
using LinearAlgebra, Statistics, BenchmarkTools, Random
```

ПРИМЕРЫ ИЗ РАЗДЕЛА 4.2

[4]:
```
# Поэлементные операции
a = rand(1:20, (4,3))
println("Сумма: ", sum(a))
println("Сумма по столбцам: ", sum(a, dims=1))
println("Сумма по строкам: ", sum(a, dims=2))
println("Произведение: ", prod(a))
println("Среднее: ", mean(a))
```

```
Сумма: 145
Сумма по столбцам: [52 44 49]
Сумма по строкам: [46; 43; 22; 34;;]
Произведение: 1015822080000
Среднее: 12.083333333333334
```

[5]:
```
# Транспонирование, след, ранг, определитель
b = rand(1:20, (4,4))
println("Транспонирование:\n", transpose(b))
println("След: ", tr(b))
println("Ранг: ", rank(b))
println("Определитель: ", det(b))
println("Обратная матрица:\n", inv(b))
```

```
Транспонирование:
[19 18 2 8; 4 8 6 16; 18 15 6 16; 3 12 16 14]
След: 47
Ранг: 4
Определитель: -14419.999999999993
Обратная матрица:
[-0.08349514563106805 0.16699029126213602 -0.08155339805825247 -0.03203883495145633; -0.1239944521497920
6 0.10513176144244117 -0.13606102635228853 0.09195561719833563; 0.1725381414701804 -0.20221914008321787
0.10208044382801669 0.01969486823855758; -0.007766990291262122 0.01553398058252426 0.08543689320388351
-0.03786407766990292]
```

Рисунок 4.1: Поэлементные операции и операции над матрицами

4.1.2 Нормы, расстояния и умножение матриц

На рис. 4.2 показано вычисление норм векторов (евклидовой и p-нормы), расстояния между векторами и угла между ними. Также продемонстрировано умножение матриц, создание единичной матрицы и вычисление скалярного произведения векторов двумя способами.

```
[6]: # Нормы и расстояния
X = [2, 4, -5]
Y = [1, -1, 3]
println("Норма X (евклидова): ", norm(X))
println("p-норма (p=1): ", norm(X, 1))
println("Расстояние между X и Y: ", norm(X - Y))
println("Угол между векторами (рад): ", acos((X' * Y) / (norm(X) * norm(Y))))
```

```
Норма X (евклидова): 6.708203932499369
p-норма (p=1): 11.0
Расстояние между X и Y: 9.486832980505138
Угол между векторами (рад): 2.4404307889469252
```

```
[8]: #Умножение матриц, скалярное произведение,
#единичная
A = rand(1:10, (2,3)); B = rand(1:10, (3,4))
println("A * B =\n", A * B)
I3 = Matrix{Int}(I, 3, 3) # Единичная матрица
println("Единичная 3x3:\n", I3)
println("Скалярное произведение dot: ", dot(X, Y))
println("Скалярное произведение X'Y: ", X'Y)
```

```
A * B =
[157 160 126 123; 154 136 105 90]
Единичная 3x3:
[1 0 0; 0 1 0; 0 0 1]
Скалярное произведение dot: -17
Скалярное произведение X'Y: -17
```

Рисунок 4.2: Нормы, расстояния и умножение матриц

4.1.3 Факторизации и спектральное разложение

На рис. 4.3 приведен пример решения СЛАУ, выполнения LU- и QR-разложения, а также проверки ортогональности матрицы Q и спектрального разложения симметризованной матрицы.

```
[9]: #Факторизации (LU, QR, Собственные)
# Решение СЛАУ
A = rand(3, 3); x = fill(1.0, 3); b = A * x
println("Решение A \\ b: ", A \ b)

# LU-разложение
Alu = lu(A)
println("L-фактор:\n", Alu.L)
println("U-фактор:\n", Alu.U)

# QR-разложение
Aqr = qr(A)
println("Матрица Q ортогональна? ", Aqr.Q' * Aqr.Q ≈ Matrix(I, 3, 3))

# Симметризация и спектральное разложение
Asym = A + A'
eig = eigen(Asym)
println("Собственные значения: ", eig.values)
println("Проверка: inv(V)*A*V =\n", inv(eig.vectors) * Asym * eig.vectors)
```

```
Решение A \ b: [1.0000000000000004, 1.0000000000000002, 0.9999999999999996]
L-фактор:
[1.0 0.0 0.0; 0.9444620603534252 1.0 0.0; 0.5977894086156186 0.24692098463403528 1.0]
U-фактор:
[0.871363426805717 0.01887965712510964 0.9920180053076475; 0.0 0.7519026218143477 -0.3083429732934895; 0
.0 0.0 0.4492847994021838]
Матрица Q ортогональна? true
Собственные значения: [-0.4156881744279469, 0.4108061385654289, 4.028750630862902]
Проверка: inv(V)*A*V =
[-0.41568817442794703 1.3877787807814457e-16 6.74807432154978e-16; -5.204170427930421e-17 0.410806138565
42925 -1.4432899320127035e-15; 5.551115123125783e-17 -1.7763568394002505e-15 4.028750630862903]
```

Рисунок 4.3: Факторизации и спектральное разложение

4.1.4 Оценка эффективности с использованием BenchmarkTools

На рис. 4.4 продемонстрировано сравнение скорости вычисления собственных значений для обычной симметричной матрицы, зашумленной матрицы и матрицы с явно указанной структурой `Symmetric`. Показано, что явное указание структуры позволяет значительно ускорить вычисления.

[10]:
```
# BenchmarkTools (спец. структуры)
n = 1000
A = randn(n,n)
Asym = A + A'
Asym_noisy = copy(Asym)
Asym_noisy[1,2] += 5eps()
Asym_explicit = Symmetric(Asym_noisy)

# Сравнение скорости
println("Обычная симметричная:")
@btime eigvals(Asym);
println("Зашумленная:")
@btime eigvals(Asym_noisy);
println("Явно указанная Symmetric:")
@btime eigvals(Asym_explicit);
```

```
Обычная симметричная:
  128.718 ms (18 allocations: 7.98 MiB)
Зашумленная:
  724.310 ms (27 allocations: 7.93 MiB)
Явно указанная Symmetric:
  125.327 ms (18 allocations: 7.98 MiB)
```

Рисунок 4.4: Оценка эффективности (BenchmarkTools)

4.2 Задания для самостоятельной работы

4.2.1 Произведение векторов (4.4.1)

На рис. 4.5 представлено вычисление скалярного произведения вектора на самого себя и внешнего произведения (матрицы).

ЗАДАНИЯ ДЛЯ САМОСТОЯТЕЛЬНОЙ РАБОТЫ

4.4.1. Произведение векторов

```
11]: v = rand(1:10, 5) # Задаем вектор
dot_v = dot(v, v) # Скалярное на себя
outer_v = v * v'  # Внешнее произведение
println("dot_v = ", dot_v)
println("outer_v =\n", outer_v)
```

```
dot_v = 313
outer_v =
[4 20 16 16 18; 20 100 80 80 90; 16 80 64 64 72; 16 80 64 64 72; 18 90 72 72 81]
```

4.4.2. Системы линейных уравнений

```
12]: println("--- С двумя неизвестными ---")
# a) x+y=2, x-y=3
println("a) ", [1 1; 1 -1] \ [2, 3])
# b) x+y=2, 2x+2y=4 (Бесконечно много решений. Ранг = 1)
println("b) Ранг: ", rank([1 1; 2 2]), " (решений бесконечно много)")
# c) x+y=2, 2x+2y=5 (Нет решений)
println("c) Ранг A: ", rank([1 1; 2 2]), ", Ранг A|b: ", rank([1 1 2; 2 2 5]), " (решений нет)")
# d) x+y=1, 2x+2y=2, 3x+3y=3 (Псевдорешение)
println("d) ", pinv([1 1; 2 2; 3 3]) * [1, 2, 3])
# e) x+y=2, 2x+y=1, x-y=3
println("e) ", pinv([1 1; 2 1; 1 -1]) * [2, 1, 3])
# f) x+y=2, 2x+y=1, 3x+2y=3
println("f) ", pinv([1 1; 2 1; 3 2]) * [2, 1, 3])
```

```
--- С двумя неизвестными ---
a) [2.5, -0.5]
b) Ранг: 1 (решений бесконечно много)
c) Ранг A: 1, Ранг A|b: 2 (решений нет)
d) [0.4999999999999999, 0.5]
e) [1.5000000000000004, -0.9999999999999998]
f) [-0.9999999999999987, 2.9999999999999982]
```

Рисунок 4.5: Произведение векторов (4.4.1)

4.2.2 Системы линейных уравнений (4.4.2)

На рис. 4.6 показано решение СЛАУ с двумя неизвестными. Рассмотрены случаи единственного решения, бесконечного множества решений (ранг матрицы равен 1), отсутствия решений и переопределенных систем (решение через псевдообратную матрицу).

[13]:
```
println("\n--- С тремя неизвестными ---")
# a) x+y+z=2; x-y-2z=3
println("a) ", pinv([1 1 1; 1 -1 -2]) * [2, 3])
# b) x+y+z=2; 2x+2y-3z=4; 3x+y+z=1
println("b) ", [1 1 1; 2 2 -3; 3 1 1] \ [2, 4, 1])
# c) x+y+z=1; x+y+2z=0; 2x+2y+3z=1
println("c) Ранг A: ", rank([1 1 1; 1 1 2; 2 2 3]), ", Ранг A|b: ", rank([1 1 1 1; 1 1 2 0; 2 2 3 1]))
# d) x+y+z=1; x+y+2z=0; 2x+2y+3z=0
println("d) Ранг A: ", rank([1 1 1; 1 1 2; 2 2 3]), ", Ранг A|b: ", rank([1 1 1 1; 1 1 2 0; 2 2 3 0]))
```

```
--- С тремя неизвестными ---
a) [2.2142857142857144, 0.35714285714285676, -0.5714285714285716]
b) [-0.5, 2.5, 0.0]
c) Ранг A: 2, Ранг A|b: 2
d) Ранг A: 2, Ранг A|b: 3
```

4.4.3. Операции с матрицами (Диагонализация, Степени и Собственные значения)

[14]:
```
# 1. Приведение матриц к диагональному виду
# a)
A_a = [1 -2; -2 1]
eig_a = eigen(A_a)
println("1a) Диагональная матрица Λ:\n", Diagonal(eig_a.values))
println("    Матрица собственных векторов V:\n", eig_a.vectors)
```

```
1a) Диагональная матрица Λ:
Diagonal([-1.0, 3.0])
    Матрица собственных векторов V:
[-0.7071067811865475 -0.7071067811865475; -0.7071067811865475 0.7071067811865475]
```

[15]:
```
# b)
A_b = [1 -2; -2 3]
eig_b = eigen(A_b)
println("\n1b) Диагональная матрица Λ:\n", Diagonal(eig_b.values))
println("    Матрица собственных векторов V:\n", eig_b.vectors)
```

```
1b) Диагональная матрица Λ:
Diagonal([-0.2360679774997897, 4.23606797749979])
    Матрица собственных векторов V:
[-0.8506508083520399 -0.5257311121191335; -0.5257311121191335 0.8506508083520399]
```

Рисунок 4.6: Системы линейных уравнений с двумя неизвестными (4.4.2)

На рис. 4.7 показано решение СЛАУ с тремя неизвестными. Аналогично проверена совместность систем с помощью рангов основной и расширенной матриц, а также найдены решения через `pinv`.

[16]:
```
# c)
A_c = [1 -2 0; -2 1 2; 0 2 0]
eig_c = eigen(A_c)
println("\n1c) Диагональная матрица Λ:\n", Diagonal(eig_c.values))
```

```
1c) Диагональная матрица Λ:
Diagonal([-2.1413361156553594, 0.5151380471280724, 3.6261980685272936])
```

[17]:
```
# 2. Вычисления (степени и корни)
# a) Возведение в степень 10
M2_a = [1 -2; -2 1]
println("2a) Матрица в 10-й степени:\n", M2_a^10)

# b) Квадратный корень из матрицы
M2_b = [5 -2; -2 5]
println("\n2b) Квадратный корень из матрицы:\n", sqrt(M2_b))
# (Проверка: sqrt(M2_b)^2 должно равняться M2_b)

# c) Кубический корень (возводим в степень 1/3)
M2_c = [1 -2; -2 1]
println("\n2c) Кубический корень из матрицы:\n", M2_c^(1/3))

# d) Квадратный корень из матрицы (несимметричной)
M2_d = [1 2; 2 3]
println("\n2d) Квадратный корень из матрицы:\n", sqrt(M2_d))
```

```
2a) Матрица в 10-й степени:
[29525 -29524; -29524 29525]

2b) Квадратный корень из матрицы:
[2.1889010593167337 -0.45685025174785665; -0.45685025174785665 2.1889010593167337]

2c) Кубический корень из матрицы:
ComplexF64[0.9711247851537039 + 0.43301270189221924im -0.47112478515370404 + 0.43301270189221924im; -0.4
7112478515370404 + 0.43301270189221924im 0.9711247851537039 + 0.43301270189221924im]

2d) Квадратный корень из матрицы:
ComplexF64[0.5688644810057828 + 0.35157758425414287im 0.9204420652599258 - 0.2172868967516401im; 0.92044
20652599258 - 0.21728689675164012im 1.489306546265709 + 0.1342906875025027im]
```

Рисунок 4.7: Системы линейных уравнений с тремя неизвестными (4.4.2)

4.2.3 Операции с матрицами (4.4.3)

На рис. 4.8 и 4.9 представлено приведение матриц 2x2 и 3x3 к диагональному виду (нахождение собственных значений и собственных векторов).

[18]:
```
# 3. Собственные значения матрицы A и оценка эффективности
# Задаем матрицу A
A = [
    140  97  74 168 131;
     97 106  89 131  36;
     74  89 152 144  71;
    168 131 144  54 142;
    131  36  71 142  36
]

# Находим собственные значения
vals = eigvals(A)
println("Собственные значения матрицы A:\n", vals)
```

```
Собственные значения матрицы A:
[-128.49322764802145, -55.88778455305688, 42.75216727931894, 87.16111477514521, 542.4677301466143]
```

[19]:
```
# Создаем диагональную матрицу из собственных значений
D = Diagonal(vals)
println("\nДиагональная матрица (из собственных значений A):\n", D)

# Создаем нижнедиагональную (нижнетреугольную) матрицу из матрицы A
# (по сути, оставляем только элементы на главной диагонали и ниже)
L = LowerTriangular(A)
println("\nНижнетреугольная матрица из A:\n", L)
```

```
Диагональная матрица (из собственных значений A):
Diagonal([-128.49322764802145, -55.88778455305688, 42.75216727931894, 87.16111477514521, 542.46773014661
43])

Нижнетреугольная матрица из A:
[140 0 0 0 0; 97 106 0 0 0; 74 89 152 0 0; 168 131 144 54 0; 131 36 71 142 36]
```

Рисунок 4.8: Приведение матриц 2x2 к диагональному виду (4.4.3)

[20]:
```
# Оценка эффективности (используем BenchmarkTools)
println("\nОценка эффективности операций:")
println("1) Умножение диагональной матрицы на себя:")
@btime $D * $D

println("\n2) Умножение нижнетреугольной матрицы на себя:")
@btime $L * $L
```

```
Оценка эффективности операций:
1) Умножение диагональной матрицы на себя:
  31.698 ns (2 allocations: 96 bytes)

2) Умножение нижнетреугольной матрицы на себя:
  204.016 ns (2 allocations: 272 bytes)
```

[20]:
```
5×5 LowerTriangular{Int64, Matrix{Int64}}:
 19600      ·      ·      ·     ·
 23862  11236      ·      ·     ·
 30241  22962  23104      ·     ·
 55955  33776  29664   2916     ·
 55658  30033  33796  12780  1296
```

4.4.4. Модель Леонтьева (продуктивность матрицы)

[21]:
```
# 1. Проверка по определению
println("--- 1. Проверка по определению ---")
# По определению: берем неотрицательный вектор y и проверяем, что решение x >= 0.
# Возьмем y = [1, 1]. Если x получится с минусом, то матрица не продуктивна.

y_test = [1.0, 1.0]

# a) A = [1 2; 3 4]
A1 = [1.0 2.0; 3.0 4.0]
E1 = Matrix{Float64}(I, 2, 2)
x1 = (E1 - A1) \ y_test
println("a) Решение x = ", x1, " -> продуктивна? ", all(x1 .>= 0))
```

```
--- 1. Проверка по определению ---
a) Решение x = [0.16666666666666666, -0.5] -> продуктивна? false
```

Рисунок 4.9: Приведение матрицы 3x3 к диагональному виду (4.4.3)

На рис. 4.10 показаны вычисления степеней и корней матриц. Возведение в 10-ю степень дало целочисленную матрицу, а извлечение квадратного и кубического корней — комплексные числа.

[22]:
```
# b) A = 1/2 * [1 2; 3 4]
A2 = (1/2) * [1.0 2.0; 3.0 4.0]
E2 = Matrix{Float64}(I, 2, 2)
x2 = (E2 - A2) \ y_test
println("b) Решение x = ", x2, " -> продуктивна? ", all(x2 .>= 0))
```

```
b) Решение x = [-0.0, -1.0] -> продуктивна? false
```

[23]:
```
# c) A = 1/10 * [1 2; 3 4]
A3 = (1/10) * [1.0 2.0; 3.0 4.0]
E3 = Matrix{Float64}(I, 2, 2)
x3 = (E3 - A3) \ y_test
println("c) Решение x = ", x3, " -> продуктивна? ", all(x3 .>= 0))
```

```
c) Решение x = [1.6666666666666665, 2.5000000000000004] -> продуктивна? true
```

[27]:
```
# Функция для проверки критериев
function check_criteria(A, name)
    n = size(A, 1)
    E = Matrix{Float64}(I, n, n)

    # Проверка 2-го критерия: все элементы (E-A)^-1 >= 0
    B = inv(E - A)
    crit2 = all(B .>= 0)

    # Проверка 3-го критерия (спектральный): max|λ| < 1
    lambdas = eigvals(A)
    max_lambda = maximum(abs.(lambdas))
    crit3 = max_lambda < 1

    println("Матрица $name:")
    println("  Собственные значения: ", lambdas)
    println("  Критерий 2 (матрица (E-A)⁻¹ >= 0): ", crit2)
    println("  Критерий 3 (спектральный max|λ| < 1): ", crit3, " (max|λ| = ", round(max_lambda, digits=3)
    println()
end
```

[27]:
```
check_criteria (generic function with 1 method)
```

Рисунок 4.10: Вычисления (степени и корни) (4.4.3)

На рис. 4.11 представлено нахождение собственных значений для заданной матрицы 5x5, создание диагональной матрицы из этих значений и выделение нижнетреугольной части исходной матрицы.

```
#2 и 3. Проверка по критериям для матриц [1 2; 3 1] и их вариаций
println("--- 2 и 3. Проверка матриц из пунктов 2 и 3 ---")

# Задаем базовую матрицу для пунктов 2 и 3
M_base = [1.0 2.0; 3.0 1.0]

# a)
A_2a = M_base
check_criteria(A_2a, "2a/3a")

# b)
A_2b = (1/2) * M_base
check_criteria(A_2b, "2b/3b")

# c)
A_2c = (1/10) * M_base
check_criteria(A_2c, "2c/3c")

# 3d. Проверка матрицы 3x3
println("--- 3d. Матрица 3x3 ---")
A_3d = [
    0.1 0.2 0.3;
    0.0 0.1 0.2;
    0.0 0.1 0.3
]

check_criteria(A_3d, "3d")
```

```
--- 2 и 3. Проверка матриц из пунктов 2 и 3 ---
Матрица 2a/3a:
  Собственные значения: [-1.4494897427831779, 3.4494897427831783]
  Критерий 2 (матрица (E-A)⁻¹ >= 0): false
  Критерий 3 (спектральный max|λ| < 1): false (max|λ| = 3.449)

Матрица 2b/3b:
  Собственные значения: [-0.7247448713915892, 1.7247448713915589]
  Критерий 2 (матрица (E-A)⁻¹ >= 0): false
  Критерий 3 (спектральный max|λ| < 1): false (max|λ| = 1.725)

Матрица 2c/3c:
  Собственные значения: [-0.14494897427831785, 0.34494897427831783]
  Критерий 2 (матрица (E-A)⁻¹ >= 0): true
  Критерий 3 (спектральный max|λ| < 1): true (max|λ| = 0.345)
```

Рисунок 4.11: Собственные значения и специальные матрицы (4.4.3)

На рис. 4.12 приведена оценка эффективности умножения диагональной и нижнетреугольной матриц. Умножение диагональной матрицы происходит значительно быстрее.

```
--- 3d. Матрица 3x3 ---
Матрица 3d:
  Собственные значения: [0.02679491924311228, 0.1, 0.37320508075688774]
  Критерий 2 (матрица (E-A)⁻¹ >= 0): true
  Критерий 3 (спектральный max|λ| < 1): true (max|λ| = 0.373)
```

Рисунок 4.12: Оценка эффективности умножения матриц (4.4.3)

4.2.4 Модель Леонтьева (4.4.4)

На рис. 4.13 и 4.14 показана проверка продуктивности матриц по определению (через решение уравнения $x - Ax = y$) и по критериям (неотрицательность обратной матрицы $(E - A)^{-1}$ и спектральный критерий $\max |\lambda| < 1$).

```
#2 и 3. Проверка по критериям для матриц [1 2; 3 1] и их вариаций
println("--- 2 и 3. Проверка матриц из пунктов 2 и 3 ---")

# Задаем базовую матрицу для пунктов 2 и 3
M_base = [1.0 2.0; 3.0 1.0]

# a)
A_2a = M_base
check_criteria(A_2a, "2a/3a")

# b)
A_2b = (1/2) * M_base
check_criteria(A_2b, "2b/3b")

# c)
A_2c = (1/10) * M_base
check_criteria(A_2c, "2c/3c")

# 3d. Проверка матрицы 3x3
println("--- 3d. Матрица 3x3 ---")
A_3d = [
    0.1 0.2 0.3;
    0.0 0.1 0.2;
    0.0 0.1 0.3
]

check_criteria(A_3d, "3d")
```

```
--- 2 и 3. Проверка матриц из пунктов 2 и 3 ---
Матрица 2a/3a:
  Собственные значения: [-1.4494897427831779, 3.4494897427831783]
  Критерий 2 (матрица (E-A)⁻¹ >= 0): false
  Критерий 3 (спектральный max|λ| < 1): false (max|λ| = 3.449)

Матрица 2b/3b:
  Собственные значения: [-0.7247448713915892, 1.7247448713915891]
  Критерий 2 (матрица (E-A)⁻¹ >= 0): false
  Критерий 3 (спектральный max|λ| < 1): false (max|λ| = 1.725)

Матрица 2c/3c:
  Собственные значения: [-0.14494897427831785, 0.34494897427831783]
  Критерий 2 (матрица (E-A)⁻¹ >= 0): true
  Критерий 3 (спектральный max|λ| < 1): true (max|λ| = 0.345)
```

Рисунок 4.13: Проверка продуктивности матриц по определению (4.4.4)

```
--- 3d. Матрица 3x3 ---
Матрица 3d:
  Собственные значения: [0.02679491924311228, 0.1, 0.37320508075688774]
  Критерий 2 (матрица (E-A)⁻¹ >= 0): true
  Критерий 3 (спектральный max|λ| < 1): true (max|λ| = 0.373)
```

Рисунок 4.14: Проверка продуктивности матриц по критериям (4.4.4)

5 Результаты

В ходе выполнения работы были получены следующие основные результаты:

1. **Базовые операции:** Успешно выполнены все базовые операции над матрицами и векторами, результаты совпадают с математическими ожиданиями.
2. **Факторизации:** Получены LU- и QR-разложения. Проверка ортогональности матрицы Q дала значение `true`. Спектральное разложение подтверждено проверкой `inv(V)*A*V`, результат которой близок к диагональной матрице.
3. **Бенчмаркинг:** Вычисление собственных значений для явно симметричной матрицы заняло ~125 мс, для обычной симметричной — ~128 мс, а для «зашумленной» матрицы — ~724 мс. Явное указание структуры `Symmetric` позволяет избежать проверок и работает так же быстро, как и с обычной симметричной матрицей.
4. **СЛАУ:** Решены системы с двумя и тремя неизвестными. Для несовместных систем ранг основной матрицы оказался меньше ранга расширенной. Для систем с бесконечным числом решений ранги совпадали.
5. **Собственные значения и диагонализация:** Найдены собственные значения для матриц 2x2, 3x3 и 5x5. Построены диагональные матрицы Λ и матрицы собственных векторов V.
6. **Степени и корни:** Возведение в 10-ю степень дало матрицу с целыми числами. Корни (квадратные и кубические) из матриц были вычислены в комплексной области.

7. **Модель Леонтьева:** Матрицы $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ и $\frac{1}{2}\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ оказались непродуктивными (решение содержит отрицательные компоненты, критерии дают `false`). Матрица $\frac{1}{10}\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ и заданная матрица 3x3 являются продуктивными (все критерии `true`).

6 Обсуждение результатов

Полученные результаты полностью согласуются с теоретическими сведениями.

- **Эффективность:** Результаты бенчмаркинга наглядно демонстрируют важность использования специализированных структур данных (например, `Symmetric`) для оптимизации вычислений. Если матрица математически симметрична, но в неё добавлен шум, компьютер трактует её как обычную, что замедляет работу в разы. Явное указание типа позволяет алгоритмам использовать свойства симметрии.
- **СЛАУ:** Использование псевдообратной матрицы `pinv` позволило найти решения для переопределенных систем, что демонстрирует гибкость Julia.
- **Корни матриц:** Появление комплексных чисел при извлечении корней из матриц объясняется тем, что не все матрицы имеют вещественные корни (спектр матрицы может содержать комплексные значения).
- **Модель Леонтьева:** Проверка продуктивности показала, что даже небольшие изменения в матрице (умножение на коэффициент) могут кардинально менять её свойства. Спектральный критерий и критерий неотрицательности обратной матрицы дали согласованные результаты.

7 Заключение

В ходе выполнения лабораторной работы были изучены возможности языка Julia и его пакетов для решения задач линейной алгебры.

Основные итоги:

1. Освоены базовые операции над матрицами и векторами, включая факторизации и спектральное разложение.
2. Проведена оценка эффективности вычислений с помощью `BenchmarkTools`, подтверждена важность использования специализированных структур данных.
3. Решены практические задачи: СЛАУ, диагонализация матриц, вычисление степеней и корней, проверка продуктивности матриц в модели Леонтьева.
4. Полученные результаты демонстрируют высокую скорость и удобство работы с линейной алгеброй в Julia.

Цель работы достигнута в полном объёме.

Список литературы

1. *The Julia Language*. Julia Documentation. — 2026. — URL: https://docs.julialang.org ; Официальная документация языка Julia.
2. *Беклемишев Д. В.* Курс аналитической геометрии и линейной алгебры. — 21-е изд., стер. — Санкт-Петербург : Лань, 2025. — С. 448. — ISBN 978-5-507-50839-6.